Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I think practicing how to write and have good coverage unit tests will help me when I start to build my helper functions and the overall project. Because it's not a project we can check with a built-in assignment, It'll be important I can properly test my functions so that I know they are working properly and as intended.

I expect to use a couple of if/elif/else statements because depending on what the user inputs the output would change. I'd also expect to use a loop in order to go through the data the user inputs to convert it into a string with everything included. So I would need to make a list of lists to contain all of the data in order to use the for loop. I'll need to use dataclasses as well so that the data with multiple attributes can be represented.

I hope to improve my ability to write routes and pages as that is probably my weakest skill at this point because there weren't a lot of opportunities to write them, and the project requires 7 routes.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I think the easiest thing for me will be to write any helper functions because I think writing functions is my strongest skill and we've had a lot of practice throughout the semester. If I get stuck I can refer to previous assignments to review and hopefully get inspiration on how/what to write.

The hardest part will be the organization and ensuring I get every element I need in the program. So far, everything we've done this semester has been guided, but this is relatively loose in terms of guidance. I have to make sure I cover all of the requirements for the project as well as everything needed for my program to run properly. I'm a little afraid that I could miss an important function/route/etc that my program will need even if I have all of the project requirements

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

Most time/effort tasks:

- Designing the website layout (routes and pages)
- Unit testing each section (ensure the proper results are given)
- Documenting each implemented function (make sure everything is clear and concise)
- Determining what functions should be included (make sure all basis are covered)
- Finding errors in the website and then locating where to fix said error (i.e. not updating the instructions when the user updates)

Features /components to implement first:

- Helper functions
- State dataclass

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Some distractions will be other coursework and assignments. I have a couple exams throughout the weeks of the project that might make it difficult to find time to do the project. I'll try to minimize this by dedicating certain times of day to work on the project because I know it'll take a lot of time to go through, so I'll allocate 1-3 hours a day

(changing based on progress) to work on the project.

I'll need to utilize past homework assignments, and example drafter websites we've seen in class in order to reference what I am doing. Having a model I can look at usually helps me when deciding on what to do, so I'll keep examples open. The past homework assignments will be there to refresh my knowledge on a certain section and to refer back to if I forget how something is written/implemented

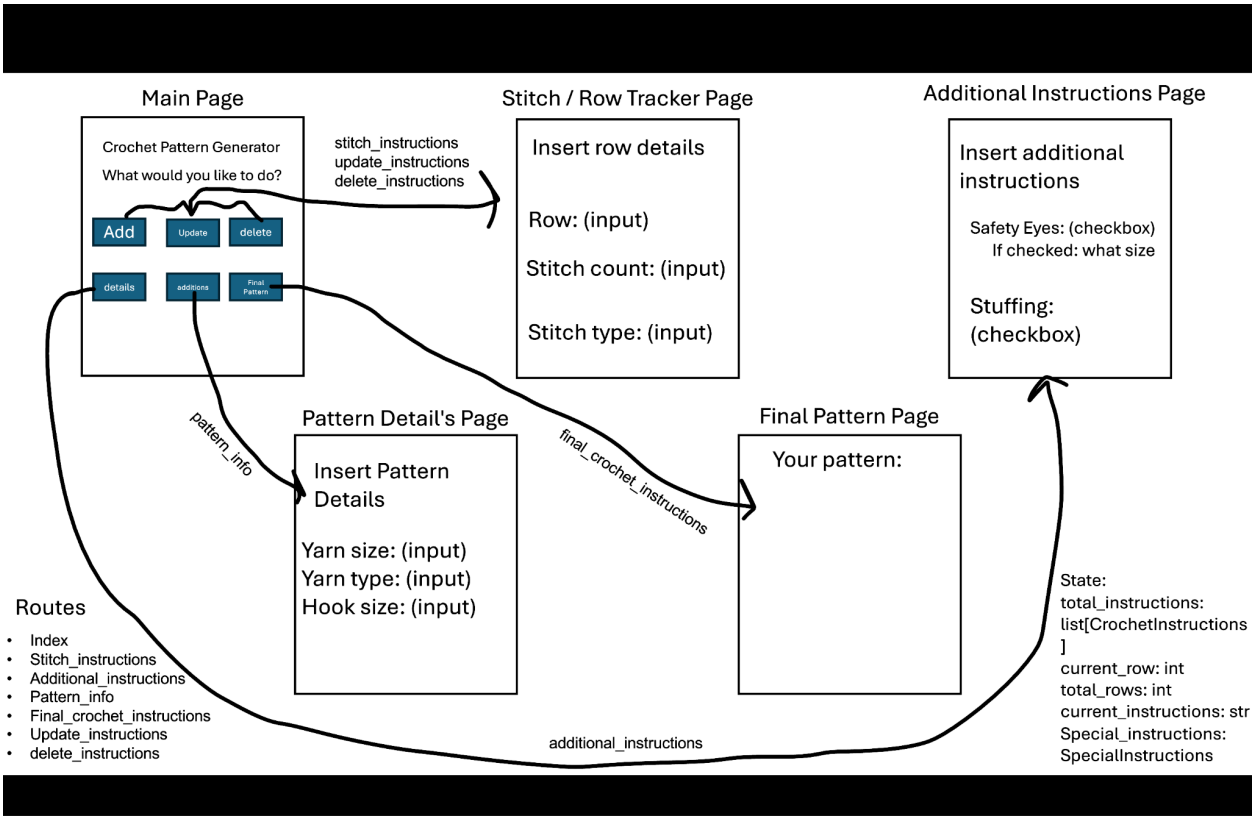
1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.



2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	**The purpose of this site is to generate a simple final crochet pattern using the inputted details from the user.**
State Dataclass Attributes Minimum of 4 attributes, one of	@ dataclass class State: total_instructions: list[CrochetInstructions]

which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	(this will be modified by adding a new list each time a new Crochetinstruction is used (as in the user makes a new row and puts in new instructions, those instructions are added to the list)) current_row: int (modified based on what row the user is on) total_rows: int (modified based on how many rows the user has created in total) current_instructions: str (updates the instructions as a string each time a new row is added) Special_instructions: SpecialInstructions (updates with what the user adds to the pattern (stuffing, safety eyes)

Second Dataclass
 	
Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)
	I'm currently planning on making three separate dataclasses
	
	@dataclass
	class CrochetInstructions:
	Row: int (will be updated depending on what row the user is currently writing (row 1 versus row 10))
	Type_of_stitch: str (updated on what kind of stitch the user is using at that time (single, increasing, decreasing,etc.))
	
	Stitch_count: int (updated final stitch count in the row (6 stitches in the row 1, 12 stitches in row 2))
	
	@dataclass
	class SafetyEyes:
	Size: int
	Row: int
	Has_safety_eyes:bool

	@ dataclass class SpecialInstructions: Safety_eyes: SafetyEyes Stuffing: bool
Routes (minimum 7) The names of your route functions	- Index - Stitch_instructions - Additional_instructions - Pattern_info - Final_crochet_instructions - Update_instructions - delete_instructions
Pages (minimum 5) A brief description of the pages in your site	- Main Page - The home page, where the user will see buttons to move to different pages - Stitch/Row Tracker Page - Where the user will input stitch and row information - Additional instructions page - Where the user will input special instructions such as if they used safety eyes and if they stuffed the project - Pattern details page - Where the user will input the details such as yarn size/type and hook size - Final pattern page - Where the user will receive/view the generated pattern
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter	- TextBox (using multiple textboxes): - Will be on the stitch tracker page and the pattern details page - Used to get what the user has done with the pattern that can then be added to a list or modified the state - Checkbox - On additional instructions page - Used to determine if the user

Documentation.	added additional details. If checked, the additional detail will be added to the pattern
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	- One - If statement to determine if any additional instructions were added, if they are false no additional instructions are added into the final pattern - Two - If statement that if the user inputs pattern details, it will be added to the final pattern if they don't add anything then the final pattern wont include that information - Three If statement that if the user inputs stitch instructions (and are valid), then they get added to a list for total_instructions to iterate through. Otherwise nothing is added
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	The loop will iterate over the total_instructions attribute in state. It will take the list and produce a string representing the instructions given in that row. The loop will add each row instructions to the string until the final pattern is produced This is meaningful because it will be what generates the working pattern for the user to view.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

the implementation of the State dataclass must have at least 4 fields.

At least one field must be a list of class instances or a dictionary of class instances.

This means that there must be a second dataclass.

At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants

can you explain this requirement for the project a little simpler

Basically, your project needs to keep track of at least 4 pieces of information that can change, and one of those pieces must be a list or dictionary containing objects created from another class.

Let's break that down!

1. You need at least 4 fields in your State dataclass

Think of State as the place where your app stores the information it needs.

For example, if you're making a study tracker, your state could look like this:

```
from dataclasses import dataclass

@dataclass
class StudySession:
    subject: str
    minutes: int

@dataclass
class State:
    sessions: list[StudySession]
    goal_minutes: int
    total_minutes: int
    current_subject: str
```

Here, State has 4 fields:

sessions — a list of study sessions

goal_minutes — the user's study goal

total_minutes — the total time studied

current_subject — the subject the user is studying

2. One field must contain objects from a second dataclass

Notice that we created a second dataclass called StudySession.

Each StudySession object stores information about one study session, such as its subject and duration.

The sessions field is a list of these objects.

For example, it could contain:

StudySession("Math", 30)

StudySession("Biology", 45)

StudySession("History", 20)

This satisfies the requirement that one field must be a list or dictionary of class instances.

3. At least 4 fields must actually change in your app

This is the part that's easy to miss!

It's not enough to create 4 fields. Your app must meaningfully modify each of at least 4 fields in one or more Drafter routes.

For example:

Field

How a route could change it

sessions

Add a new study session

goal_minutes

Let the user update their study goal

total_minutes

Update the total when a session is added

current_subject

Let the user select a different subject

A route is one of the functions in your Drafter app that handles a page or an action, such as adding a session or updating a goal.

A field that you set once and never change doesn't count as a meaningfully modified field. The fields also shouldn't just be constants that stay the same throughout the app.

The easiest way to remember it

4 or more fields in your State dataclass.

A second dataclass to represent an individual object.

A list or dictionary of those objects inside State.

At least 4 fields that your routes actually change.

You don't need to make the project complicated. Just choose information that naturally changes as someone uses your app.

Tip: Start by asking yourself, "What information does my app need to remember, and what actions will users take that change that information?" That will help you design your State dataclass correctly.

Okay. I am designing a website for users to input certain information that will then produce a completed crochet pattern.

My current draft:

5 pages:

main page (will be the page that user first sees. Four buttons that connect to the other pages depending on what action they would like to do)

stitch instruction page (will have input boxes and buttons (have not determined how many/exactly what yet) that will let the user put in the details of the pattern regarding what they have done (i.e 6 increase stitches, 12 total stitch count)

special instructions (page where user inputs special instructions as in if they added safety eyes of when they started to stuff the project things such as that)

pattern breakdown (page where user inputs pattern details like what size hook they used, what kind of yarn, etc.)

crochet instructions (page that takes all of the inputs from each of the three pages recieving input and makes the pattern)

7 Routes:

index: function that produces the main page

stitch instructions: function to move to stitch instruction page

special instructions: function to move to special instructions page

pattern info: function to move to pattern info page

crochet instructions: function to move to crochet functions page

back?: function that returns to the previous page
about?: function that explains how the website works?

State requirement:

total_stitches : int
current_row : int
total_instructions: list[CrochetInstructions]
current_instructions: str

second dataclass:

CrochetInstructions
stitch: int
row: int
type_of_stitch: str

Am I on the right track with these requirements?

…
Yes! You're definitely on the right track. Your crochet pattern generator has a clear purpose, and your idea naturally fits the State and second dataclass requirements. I'd make a few adjustments now so you can meet the requirements without making your project more complicated than it needs to be.

1. Your State dataclass
You currently have four fields, which is a good start. Let's look at each one.

Your field

Does it work?

Why?

total_stitches: int

Yes, with a small clarification

It should change when the stitch count changes.

current_row: int

Yes

It can change when the user adds or moves to another row.

total_instructions: list[CrochetInstructions]

Yes!

It stores a list of instances of your second dataclass.

current_instructions: str

Yes, if you update it

It could hold the generated pattern text and change as the pattern is edited or regenerated.

The main thing to remember: having four fields is only half the requirement. You also need to meaningfully modify all four in at least one route.

For example, when the user adds a new row of crochet instructions, a route might:

Add a new CrochetInstructions object to total_instructions.

Update total_stitches.

Update current_row.

Update current_instructions if the displayed pattern needs to be regenerated.

That could potentially update all four fields in a single route, provided each change makes sense for your app.

2. Your second dataclass

Your CrochetInstructions dataclass is a good idea, but I would reconsider what each field represents.

You currently have:

stitch: int

row: int

type_of_stitch: str

The potential issue is that stitch: int could mean either the number of stitches or the position of a stitch. You should make that more specific.

For example, a row might contain 6 increase stitches and have 12 stitches in total. Those are different pieces of information.

Here's one possible design:

```
from dataclasses import dataclass

@dataclass
class CrochetInstructions:
    row: int
    type_of_stitch: str
    stitch_count: int
```

This represents an instruction such as "Row 3: 6 single crochet stitches."

You could also add an instruction field if you want to store the exact text that should appear in the final pattern.

The best design depends on whether you're recording individual stitches, groups of stitches, or complete row instructions. For an introductory project, storing groups of stitches or row instructions is likely simpler.

3. How the whole State could look

Here's a possible version that stays close to your original plan.

```
@dataclass
class State:
    total_stitches: int
    current_row: int
    total_instructions: list[CrochetInstructions]
    current_instructions: str
```

You don't necessarily need to add more fields just to satisfy the minimum. Your existing four can work, as long as they're all meaningfully modified.

One important design decision: decide whether total_stitches represents the stitch count for the current row or the total number of stitches across the whole project. For crochet patterns, the current row's stitch count will often be more useful.

4. Your pages and routes

Your five pages make sense for the user experience. You describe seven routes, but you've only listed six route names, including about?. You may want to check that count.

Also, you don't necessarily need a separate route for every page transition if you can combine related functionality.

Here's how I'd organize the routes:

Route	Purpose
index	Display the home page
stitch_instructions	Record stitch and row details
special_instructions	Record safety eyes, stuffing notes, and other details
pattern_info	Record yarn, hook size, and other materials
crochet_instructions	Generate and display the completed pattern
back	Return to a previous page, if needed
about	Explain how the website works

The exact route names depend on your instructor's requirements and how you

implement navigation in Drafter.

5. One important consideration for your generator

Since Drafter runs Python in the browser and doesn't have its own backend or database, you should plan to keep the user's pattern data in the app's state while they're using the site. If you want their pattern to remain available after they leave or refresh the page, you could investigate browser local storage or a file download feature.

Also, your generator will need a clear rule for turning inputs into a pattern. For example, it could take the saved row instructions and format them into readable text. You don't need to build an AI-powered crochet designer to meet your goal; a well-organized pattern formatter is a much more manageable first version.

My recommendation: Keep your five-page idea and your four State fields for now. Refine what CrochetInstructions represents, then identify the route or routes that will modify each field. That will give you a strong starting point for planning the implementation.

Okay. So I think my state and CrochetInstructions are doing essentially the same thing in my initial draft so i have to adjust those.

I'm thinking i remove the row attribute in crochetinstructions and use that in the state. so my crochetinstructions only has type_of_stitch and stitch count. I'll then adjust the new state fields to be

total_instructions: list[CrochetInstructions] (get the stitch count and stitch type)

current_row: int (what row the user is currently on)

total_rows: int (how many rows in total are in the pattern)

current_instructions: str (the current instructions generated by what the user has inputted)

Yes! I think this is a good adjustment. You're separating the information about individual stitch instructions from the overall progress and generated text for the pattern. That's exactly the kind of distinction you want to make when designing your dataclasses.

I'd keep your revised design, with one small thing to think about.

1. Your CrochetInstructions dataclass

This represents an individual instruction or group of stitches.

```
@dataclass
class CrochetInstructions:
    type_of_stitch: str
    stitch_count: int
```

For example, you could create objects representing:

CrochetInstructions("single crochet", 6)

CrochetInstructions("increase", 3)

CrochetInstructions("slip stitch", 1)

Each object describes a stitch type and how many of that stitch to use. Your list can hold multiple objects.

2. Your revised State dataclass

Your four proposed fields make sense:

```
@dataclass
class State:
    total_instructions: list[CrochetInstructions]
    current_row: int
    total_rows: int
    current_instructions: str
```

Here's what each field does:

Field

Purpose

total_instructions

Stores the individual stitch instructions the user enters.

current_row

Tracks the row the user is currently working on or entering instructions for.

total_rows

Tracks the total number of rows in the pattern.

current_instructions

Stores the formatted instruction text generated from the user's inputs.

3. Does this meet the project requirements?
Potentially, yes! Here's how it lines up with the requirements you shared.

At least 4 fields in State

You have exactly four.

A second dataclass

CrochetInstructions is separate from State.

A list of class instances

total_instructions is a list of CrochetInstructions objects.

All four fields must be meaningfully modified

You still need to implement routes that actually change each field. Merely declaring the fields doesn't satisfy this part.

4. One design question to consider

Your CrochetInstructions objects currently don't identify which row they belong to.

Imagine your user enters:

Row 1: 6 single crochet stitches

Row 2: 6 increase stitches

Row 3: 12 single crochet stitches

If all your objects are stored in one list, how will your program know which instructions belong to Row 1, Row 2, or Row 3?

You have two reasonable options:

Option A: Keep your design simple and store the instructions for the current row, then format or save that row before moving to the next one.

Option B: Add a row field back to CrochetInstructions, so every instruction knows which row it belongs to.

Neither is automatically wrong. Option A could be simpler for a first project, but you'll need a clear plan for retaining earlier rows rather than accidentally replacing their instructions.

My recommendation: Your revised design is a solid starting point. Before finalizing it, decide how you will store instructions for multiple rows. That decision will help determine whether your current four fields are enough for the functionality you want.

can i add the row field back to the crochetinstructions and then access current_row in state seperate from the total_instructions? So i'd access the list twice?

Yes, absolutely! And you're thinking about it in a useful way. You can have row inside CrochetInstructions and current_row separately inside State. They serve different purposes.

1. What the two fields mean

For example, your dataclasses could look like this:

```
@dataclass
class CrochetInstructions:
    row: int
    type_of_stitch: str
    stitch_count: int

@dataclass
class State:
    total_instructions: list[CrochetInstructions]
    current_row: int
    total_rows: int
    current_instructions: str
```

Here's the distinction:

CrochetInstructions.row tells you which row a particular instruction belongs to.

State.current_row tells you which row the user is currently working on.

They're related, but they're not the same thing!

2. Do you access the list twice?

You can access the list multiple times, but you don't necessarily have to.

Imagine total_instructions contains these objects:

Row

Stitch type

Stitch count

1

Single crochet

6

1

Increase

3

2

Single crochet

9

3

Increase

4

If state.current_row is 2, you can loop through the list and select only instructions belonging to Row 2.

```
for instruction in state.total_instructions:
    if instruction.row == state.current_row:
```

print(instruction.type_of_stitch)
print(instruction.stitch_count)

This checks the list and processes only the instructions for the current row.

You could also loop through the same list separately when generating the completed pattern. That's completely fine!

3. How the fields work together

State

current_row = 2

The user is currently entering instructions for Row 2.

total_instructions list

Row 1: Single crochet, 6

Row 1: Increase, 3

Row 2: Single crochet, 9

Row 3: Increase, 4

Result

Only the instructions belonging to Row 2 are selected for the current row.

4. One thing to remember for your project requirement

Your State still has four fields, and your list contains instances of your second dataclass. That part is good.

You'll also need routes that meaningfully modify at least four fields. For example,

adding a stitch instruction could add an object to total_instructions, while moving to the next row could update current_row.

One detail: don't assume total_rows automatically updates just because you change current_row. You'll need to decide when and how your app updates the total number of rows.

My recommendation: Keep row in CrochetInstructions. It makes organizing and generating a multi-row crochet pattern much easier, while current_row in State tracks the user's current position. You aren't duplicating the same information unnecessarily; you're using two fields for two different jobs.

okay then to make sure I understand how my routes should work I changed my last two because I forgot I wanted them to be able to update as they go if they made changes to the pattern.

index - home page (gives the user the options for what they'd like to add to the pattern)

stitch_instructions - records stitch and row details (this should go to a different page for the user to input the data, updates state total_instructions, current_row, and total_row)

special_instructions - records additional instructions (this should also be another page for user to put in the data, updates current_instructions)

pattern_info - record pattern details (separate page for user to input data)

crochet_instructions - generate new pattern (separate page, updates current_instructions)

Update_instructions - allows user to go to a previous row and update the row details (takes the user back to the stitch_instruction page)

delete_instructions - allows user to remove a row (updates current_row and total_rows)

…

You're getting much closer! Your routes now have a clear purpose, and I like that you're planning to let users edit their patterns instead of forcing them to start over. I'd make a few adjustments to how you think about the routes and which state fields they update.

1. Reviewing your seven routes

1. index

Displays the home page with options to add or edit pattern information.

State changes: None required.

2. stitch_instructions

Displays a form for entering stitch types, stitch counts, and row information. When the user submits the form, it adds or updates a CrochetInstructions object in total_instructions.

State changes: total_instructions, current_row, and potentially total_rows.

3. special_instructions

Displays a form for notes such as when to attach safety eyes or add stuffing.

State changes: Your app needs somewhere to store these notes. See the important adjustment below.

4. pattern_info

Displays a form for the hook size, yarn type, and other pattern details.

State changes: The relevant pattern information fields.

5. crochet_instructions

Generates and displays the formatted crochet pattern from the information entered on the other pages.

State changes: current_instructions, when the pattern is generated or regenerated.

6. update_instructions

Lets the user select a previous row, edit its instructions, and save the changes.

State changes: Updates the relevant object in total_instructions. Regenerate the displayed pattern afterward.

7. delete_instructions

Removes a selected row's instructions from the list.

State changes: total_instructions, total_rows, and possibly current_row if the current row is removed.

2. Three things I'd adjust

A. Special instructions need their own storage

You currently plan for special_instructions to update current_instructions. However, that field is meant to hold the generated pattern text, not necessarily the raw notes the user enters.

It would be clearer to have a separate field, such as:

special_notes: list[str]

Then your app can store the user's notes in special_notes and combine them with the stitch instructions when generating current_instructions.

B. Pattern information also needs to be stored

The pattern_info route needs somewhere to save the yarn type, hook size, and other details. You could use separate fields in State, or a separate dataclass for the pattern's metadata if that makes sense for your design.

You don't have to add all those fields immediately, but you should plan where that information will live.

C. Editing and deleting should change the list

This is the most important part of your editing plan.

Suppose your list contains:

Row 1: 6 single crochet stitches

Row 2: 12 increase stitches

Row 3: 18 single crochet stitches

If the user edits Row 2, you want to replace the existing Row 2 instructions, not add another Row 2 to the list.

Similarly, deleting Row 2 should remove its instructions from the list. You'll then need to decide whether the remaining rows should be renumbered and whether total_rows should decrease.

3. How this relates to your State requirement

Your current four fields were:

total_instructions: list[CrochetInstructions]

current_row: int

total_rows: int

current_instructions: str

Here's a possible way to satisfy the requirement that all four fields be meaningfully modified:

Field

Example of a meaningful change

total_instructions

Add, edit, or delete a row's instructions

current_row

Change the selected row when the user navigates between rows

total_rows

Increase or decrease the count when rows are added or deleted

current_instructions

Regenerate the displayed pattern after an edit

One caution: don't change a field just to satisfy the requirement. Each change should make sense for the app. For example, navigating between rows should only change current_row if your design actually tracks the selected row.

4. A suggested workflow

Home page

Choose what to add or change

Input pages

Stitches · Special notes · Pattern details

Save changes to State

Add, update, or remove saved data

Generate or regenerate pattern

Combine saved information into readable crochet instructions

The key idea is that your input pages collect information, while the routes that process submissions save or modify that information. Your generator then uses the saved information to create the pattern.

My recommendation: Keep your seven routes as a starting point, but plan to store special notes and pattern details separately from the generated pattern text. That will make editing much easier and help your State accurately represent everything your app needs to remember.

okay so:

make another state attribute special notes: list[str]

make another dataclass for special instructions. I think I'll need another dataclas for this section because of all of the details for the special instructions. I might be missing some things that can be added to a pattern for special instructions but generally something like this

class SafetyEyes:

    size: int

    row: int

    has_safety_eyes: bool

class SpecialInstructions

    safety_eyes: SafetyEyes

    stuffing: bool

I think you're thinking about this correctly! A separate dataclass for special instructions makes sense because safety eyes, stuffing, and other assembly details have different properties from your stitch instructions.

I'd make two small adjustments to your plan.

1. Your SafetyEyes dataclass

Your idea is good:

@dataclass

class SafetyEyes:

    size: int

    row: int

    has_safety_eyes: bool

Each field has a distinct purpose:

size: The size of the safety eyes.

row: The row where the eyes should be placed.

has_safety_eyes: Whether the project uses safety eyes at all.

One thing to consider: if has_safety_eyes is False, the size and row won't really matter. That's okay for a beginner project, but your app should only include eye placement instructions when the user selects that they are using safety eyes.

2. Your SpecialInstructions dataclass

You could organize it like this:

@dataclass

class SpecialInstructions:

    safety_eyes: SafetyEyes

    stuffing: bool

This means a SpecialInstructions object contains a SafetyEyes object and a Boolean indicating whether the project needs stuffing.

For example, conceptually, your data might represent:

Safety eyes: yes, 8 mm, placed at Row 6.

Stuffing: yes.

This is an example of composition: one dataclass contains an instance of another dataclass.

3. How should this fit into State?

You suggested adding special_notes: list[str]. That works if you want users to enter free-form notes, such as "leave a long tail for sewing the arms on."

However, you now have two different kinds of special instructions:

Structured instructions: safety-eye size, placement row, and whether stuffing is needed.

Free-form notes: extra details the user types themselves.

You can support both if you want. For example:

```
@dataclass
class State:
    total_instructions: list[CrochetInstructions]
    current_row: int
    total_rows: int
    current_instructions: str
    special_instructions: SpecialInstructions
    special_notes: list[str]
```

This is a possible design, not a requirement to use exactly these fields. Notice that special_instructions now stores an object rather than a list of strings.

One important detail: Your project requires at least four state fields to be meaningfully modified by routes. Adding fields is fine, but you'll still need to make sure your routes actually update the relevant data.

4. Are you missing other special instructions?

Here are some possibilities you could consider, without needing to implement all of them.

Detail

Example

Safety eyes

Size and placement row

Stuffing

Whether to stuff the project

Sewing parts together

When to attach arms, legs, ears, etc.

Color changes

Which row to change yarn color

Closing the project

When to close the final opening

Extra notes

Any custom instruction from the user

I'd recommend keeping your first version limited to safety eyes, stuffing, and free-form notes. You can add more later if the basic version works.

One last design question: will your app let users create different kinds of crochet projects, some with safety eyes and some without? If so, your current has_safety_eyes

field is useful because it lets the generator decide whether to include that instruction.

the pattern could have safety eyes it could not depends on what the user is making. same with stuffing

so i dont need another field in the state to store pattern detials and special notes I can just use a variable?

Great question! And yes, you're right that not every crochet project will use safety eyes or stuffing.

You don't need a separate State field for every piece of information. However, there's an important difference between using a variable and storing information in State.

1. When can you use a regular variable?

A regular variable is fine for temporary information that you only need while a function is running.

For example, if a route calculates how many stitches are in a row, it could use a temporary variable to do that calculation.

But if the user enters their yarn type on one page and then navigates to another page, you need that information to remain available. A regular local variable inside the first route won't reliably preserve it for the next route.

In Drafter, put information in State when other routes need to access or update it.

2. What about pattern details and special instructions?

You have a few options:

Information

Where it could go

Stitch and row instructions

total_instructions: list[CrochetInstructions]

Safety eyes and stuffing choices

A SpecialInstructions object in State

Extra notes

special_notes: list[str] in State

Yarn type and hook size

Fields in State, or a separate dataclass stored in State

Generated pattern text

current_instructions: str in State

You could combine related information into dataclasses instead of creating lots of separate fields. That can help keep your design organized.

3. How to handle optional safety eyes and stuffing
Your current design can handle either choice:

```
@dataclass
class SafetyEyes:
    size: int
    row: int
    has_safety_eyes: bool

@dataclass
class SpecialInstructions:
    safety_eyes: SafetyEyes
    stuffing: bool
```

If the user doesn't want safety eyes or stuffing, the Boolean fields can be False, and your pattern generator can skip those instructions.

You could also use optional values for the eye size and placement when safety eyes aren't being used, but you don't need to complicate things until you know how you want the input form to behave.

4. My recommendation for your project

Keep your original four required State fields, but add a field for each additional group of information that needs to survive between routes. For example, one field could store all the special-instruction data in a SpecialInstructions object, and another could store pattern details in a separate dataclass.

That way, you aren't adding a separate state field for every single detail, and your data remains accessible when the user navigates between pages.

The rule of thumb: If another route needs the information later, store it in State (directly or inside an object held by State). If it's only a temporary calculation, a regular variable is fine.

okay so my updated state would be

total_instructions: list[CrochetInstructions] <- route: stitch_instructions, delete_instructions, update_instructions

current_row: int <- route: stitch_instructions, delete_instructions

total_rows: int <- route:stitch_instructions, delete_instructions

current_instructions:str <- route: current_instructions

special_instructions: SpecialInstructions <- route: special_instructions

Yes! This is looking much more organized. You've identified five state fields and connected them to the routes responsible for changing them. You're very close to having a solid plan.

I'd make two small corrections to your route mapping.

1. Reviewing your updated State

State field

Routes that modify it

My feedback

total_instructions: list[CrochetInstructions]

stitch_instructions, delete_instructions, update_instructions

Correct. These routes add, remove, or edit crochet instructions.

current_row: int

stitch_instructions, delete_instructions

Good. Consider whether update_instructions also needs to change it when the user selects a different row to edit.

total_rows: int

stitch_instructions, delete_instructions

Correct. Increase or decrease the count when rows are added or removed.

current_instructions: str

crochet_instructions

Correct, assuming this route generates or regenerates the pattern text.

special_instructions: SpecialInstructions

special_instructions

Correct. This route can update the safety-eye and stuffing choices.

2. Two details to think about

First: updating current_instructions.

When a user edits or deletes a row, the generated pattern becomes outdated. You have two reasonable options:

Regenerate current_instructions inside the edit and delete routes.

Regenerate it when the user visits the crochet_instructions route.

The second option may be simpler for your project. Just make sure the displayed pattern is regenerated from the latest saved data.

Second: updating current_row.

You don't necessarily need to change current_row when editing a row. If you use it to track which row is selected, though, your update route might change it when the user chooses a different row.

3. Does this satisfy the project requirement?

Yes, your plan is on track:

You have at least four state fields.

CrochetInstructions and SpecialInstructions are separate dataclasses.

total_instructions is a list of instances of CrochetInstructions.

You have identified routes that can meaningfully modify at least four state fields.

One final distinction: listing a route beside a field doesn't automatically satisfy the requirement. Your implementation must actually modify the field's value in that route.

For example, delete_instructions must really remove an object from total_instructions

and update total_rows when appropriate.

Overall, I'd keep this design. It's organized, realistic for a CS1 project, and directly connected to the functionality you want your crochet pattern generator to provide.

I also need three if statements that meaningfully change the logic of the site. These were the three I came up with, do they meet the project requirements and do they make sense in the context of the site:

If statement to determine if any additional instructions were added, if they are false no additional instructions are added into the final pattern

If statement that if the user inputs pattern details, it will be added to the final pattern if they don't add anything then the final pattern wont include that information

If statement that if the user inputs stitch/row details then those details are added into a list

Yes! Your three ideas make sense for a crochet pattern generator, and the first two are good examples of conditional logic. I'd adjust the third one slightly because, as you currently describe it, it sounds more like an action than a decision.

The key requirement is that your if statements should meaningfully change how your website behaves, rather than simply checking something without affecting the result.

1. Checking for additional instructions

Your idea: If no additional instructions are added, don't include them in the final pattern.

Good conditional

This makes sense because not every crochet project needs extra notes, such as when to attach safety eyes or when to stuff the project.

For example, conceptually:

```
if special_notes:
    # Include the additional notes in the pattern
```

If the list is empty, the condition is false, so the notes are skipped.

2. Checking for pattern details

Your idea: If the user enters pattern details, include them in the final pattern; otherwise, leave them out.

Good conditional

This also makes sense. Some users might enter their yarn type, hook size, and other materials, while others might leave those fields blank.

For example:

```
if yarn_type:
    # Include the yarn type in the pattern
```

You could use similar checks for hook size and other optional details.

One thing to remember: if you store several pattern details, you might need several conditions, or one condition that checks whether any of the details were entered.

3. Checking for stitch and row details

Your idea: If the user enters stitch and row details, add those details to a list.

Needs a small adjustment

The reason is that adding instructions to a list is an action, but your if statement needs to make a decision.

You could instead make the condition:

If the user has entered valid stitch instructions, add them to total_instructions. Otherwise, don't add anything.

For example:

```
if type_of_stitch and stitch_count > 0:
    # Create and add a CrochetInstructions object
```

This means the app only adds an instruction when the user has provided a stitch type and a positive stitch count.

This is a meaningful conditional because it prevents incomplete or invalid instructions from being added to the pattern.

My recommendation

Your three conditionals are a good starting point. Here's how they fit together:

Conditional

What changes in the website?

Additional notes exist

The final pattern includes the notes, or omits them.

Pattern details exist

The final pattern includes the available materials information, or omits it.

Valid stitch instructions exist

The app adds a CrochetInstructions object to the list, or doesn't add one.

One final tip: You could also consider a conditional for your SpecialInstructions dataclass: if has_safety_eyes is true, include the eye size and placement instructions; otherwise, omit them. That's especially relevant to your project because different crochet projects have different requirements.

You only need three meaningful if statements if that's what your assignment specifies, so you have enough ideas to choose from. Just make sure each one actually affects

the program's behavior in your implementation.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.